
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON VEIN - Instructor Solution Key

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

The task and criterion headings in this key are word-for-word identical to `ACT-III-R.md`, so a student can match each criterion one to one.

Artifact Identity

The instructor-issued artifact hashes are:

```
ACT-III.bin      a0f9206fdc4a33ecd24db15d179c31c40ec8abbc358489fe61a7d45dcf2d2b48
ACT-III.uf2      aed2e62f127f0ceb9d60621ef02c16850669679d1939f9ec37269cb7bd18aff4
ACT-III_fixed.bin bac3e278ce9ba9d0c166065b0218e71f3e3eb2ff0f80fee5f439e93ab058fa0c
ACT-III_fixed.uf2 fb53ca66ec00c448bb28f14a8bc9d9bb71161f8b448c8b8c13cd8a9b00ea7653
```

Machine check: `python scripts/verify_ctf.py` returns 10/10 checks passed against the shipped and corrected images. It asserts the four byte pairs, that only those four offsets differ, and the `ACT-III.bin` and `ACT-III_fixed.bin` SHA-256 values. Both `.bin` images are 50,732 bytes and both `.uf2` images are 102,400 bytes.

The four sabotage sites (summary):

Defect	Function	File offset	VA	Compromised	Correct
1 Beacon	<code>implant_tick</code>	<code>0xA245</code>	<code>0x1000A245</code>	<code>0xB9</code>	<code>0xB1</code>
2 Logic bomb	<code>implant_handle_command</code>	<code>0xA30B</code>	<code>0x1000A30B</code>	<code>0xB9</code>	<code>0xB1</code>
3 Persistence	inlined <code>implant_persist</code> in <code>implant_tick</code>	<code>0xA29F</code>	<code>0x1000A29F</code>	<code>0xD1</code>	<code>0xD0</code>
4 Valve auth	<code>control_handle_frame</code>	<code>0x74E9</code>	<code>0x100074E9</code>	<code>0xD1</code>	<code>0xD0</code>

Task 1: Setup and Initial Analysis (10 points)

Solution

Ghidra Setup. Import `ACT-III.bin` as Raw Binary, language ARM Cortex 32 little endian default, base address `0x10000000`, then run auto-analysis. The Ghidra project name is `IronVein_Investigation`. Because every defect is a same-size in-place byte patch, the file offset and the VA differ by exactly `0x10000000` (`VA = offset + 0x10000000`).

Vector Table Decoding. First 32 bytes of `ACT-III.bin`:

```
00 20 08 20 5D 01 00 10 1B 01 00 10 1D 01 00 10
11 01 00 10 11 01 00 10 11 01 00 10 11 01 00 10
```

Evidence	Answer
Vector table base	<code>0x10000000</code>
Initial SP	<code>0x20082000</code>
Reset handler (as stored)	<code>0x1000015D</code>
Reset instruction address	<code>0x1000015C</code>

The stored reset handler address has bit 0 set, selecting Thumb mode. Clearing bit 0 gives the real entry `0x1000015C`.

Entry and Monitor Loop. From `ACT-III-main-disasm.txt`:

```

10000234 <main>:
10000234:    b508          push    {r3, lr}
10000236:    f003 fa93     bl      10003760 <stdio_init_all>
1000023a:    4807          ldr     r0, [pc, #28] @ (10000258 <main+0x24>)
1000023c:    f003 fada     bl      100037f4 <__wrap_puts>
10000240:    f006 f8e2     bl      10006408 <monitor_init>
10000244:    b110          cbz     r0, 1000024c <main+0x18>
10000246:    f006 f9a9     bl      1000659c <monitor_step>
1000024a:    e7fc          b.n     10000246 <main+0x12>

```

Element	Address
main	0x10000234
monitor_init	0x10006408
monitor_step	0x1000659C

Module Map. Anchors for the stripped image:

Module	Anchor function	Address
Entry	main	0x10000234
Monitor / SCADA state machine	monitor_init	0x10006408
Monitor / SCADA state machine	monitor_step	0x1000659C
Control (sealed command path)	control_handle_frame	0x10007484
Valve authorization	valve_auth_apply	0x10007618
Valve	valve_apply_command	0x1000752C
Valve	valve_close	0x1000A724
Implant	implant_tick	0x1000A228
Implant	implant_handle_command	0x1000A304
Implant	implant_init	0x1000A344
Crypto	crypto_aead_seal	0x10007730
Crypto	crypto_aead_tag_equal	0x100076E4
Envelope	envelope_open_hex	0x100078C0
Radio	radio_send_frame	0x1000A41C

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project <code>IronVein_Investigation</code> , raw binary import
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor
[DOCUMENT] Base address set to 0x10000000	2	Base <code>0x10000000</code>
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base <code>0x10000000</code> , initial SP <code>0x20082000</code> , reset handler <code>0x1000015D</code>
[DOCUMENT] main and the SCADA controller state machine (monitor_step) addresses identified	1	<code>main</code> <code>0x10000234</code> , <code>monitor_step</code> <code>0x1000659C</code>
[DOCUMENT] Module map identifies the valve, control, valve_auth, implant, and monitor anchors	1	At least one correct anchor per module

Instructor Notes & Assembly

- Confirm the Ghidra import used `Raw Binary` , `ARM Cortex 32 little endian default` , base `0x10000000` , and that auto-analysis completed before any address was read. In the language dialog the student must search `Cortex` and pick the ARM Cortex 32 little endian default entry.
- Accept either the Import Results Summary or the Program Information window as proof of the name, language, and base address.
- The stored reset handler `0x1000015D` is odd because bit 0 selects Thumb; clearing it gives `0x1000015C` .
- Always say `reset handler` , never `reset pointer` .
- The vector table is identical in the compromised and corrected images because no defect touches it.
- The module map is graded on coverage, not on exhaustive function recovery: one correctly named anchor per module is sufficient.

Task 2: Bug #1 The Beacon (20 points)

Solution

Locate the branch. In `implant_tick` (starts at `0x1000A228`) the beacon gate is at file offset `0xA245` (VA `0x1000A245`). The corrected image is:

```

1000a240:    4a28        ldr     r2, [pc, #160] @ (1000a2e4 <implant_tick+0xbc>)
1000a242:    7812        ldrb    r2, [r2, #0]
1000a244:    b10a        cbz     r2, 1000a24a <implant_tick+0x22>
1000a246:    075a        lsls   r2, r3, #29
1000a248:    d009        beq.n   1000a25e <implant_tick+0x36>
1000a24a:    4d27        ldr     r5, [pc, #156] @ (1000a2e8 <implant_tick+0xc0>)
```

The beacon body itself, reached only when the gate passes:

```

1000a25e:    25c7      movs    r5, #199      @ 0xc7
1000a260:    4a23      ldr     r2, [pc, #140] @ (1000a2f0 <implant_tick+0xc8>)
1000a262:    f88d 5006 strb.w  r5, [sp, #6]
1000a266:    9200      str     r2, [sp, #0]
1000a268:    f8ad 3004 strh.w  r3, [sp, #4]
1000a26c:    f8dc 3df0 ldr.w   r3, [ip, #3568] @ 0xdf0
1000a270:    4d1d      ldr     r5, [pc, #116] @ (1000a2e8 <implant_tick+0xc0>)
1000a272:    f013 0303 ands.w  r3, r3, #3
1000a276:    bf18      it      ne
1000a278:    2301      movne   r3, #1
1000a27a:    4669      mov     r1, sp
1000a27c:    2208      movs    r2, #8
1000a27e:    481d      ldr     r0, [pc, #116] @ (1000a2f4 <implant_tick+0xcc>)
1000a280:    f88d 3007 strb.w  r3, [sp, #7]
1000a284:    f000 f8ca bl      1000a41c <radio_send_frame>

```

Instruction decode. `ldr r2, [pc, #160]` loads the neutralization gate byte at `0x20013CF1`, `ldrb r2, [r2, #0]` reads it, and the branch at `0x1000A244` decides whether the gate is clear. The correct code sends no beacon when the gate is clear, so the branch at `0x1000A244` must be `cbz (0xB1)` to the `0x1000A24A` path, which skips the send. When the gate is set and the low three bits of the tick counter roll over (`lsls r2, r3, #29` then `beq.n`), the frame at `0x1000A25E` is built. The magic word is the literal at `0x1000A2F0`, the bytes `DE AD BE EF`, and the send is the call to `radio_send_frame` at `0x1000A284`. The condition byte is the high byte at `0x1000A245`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A245</code>	<code>0xA245</code>	<code>0xB9</code>	<code>cbnz r2, 0x1000A24A</code>	<code>0xB1</code>	<code>cbz r2, 0x1000A24A</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA245</code>	<code>0x1000A245</code>	<code>0A B9</code>	<code>0A B1</code>

Why the beacon now stays silent. Under the compromised `cbnz`, the beacon gate is inverted: the send path is taken exactly when the neutralization gate is clear, so the `DE AD BE EF` frame transmits every 8 ticks. After the patch, `cbz` skips the send while the gate is clear, and the covert frame never leaves the radio. A periodic preamble is trivial to detect with a passive receiver or a breakpoint on `radio_send_frame`; once the branch is corrected, the same observation returns nothing every 8 ticks.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the beacon gate branch at <code>0x1000A245</code>	5	Address and function identified
[DOCUMENT] Documented <code>cbz</code> versus <code>cbnz</code> and the every-8-ticks <code>DE AD BE EF</code> beacon	5	Correct branch semantics and beacon detail
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so the beacon gate is closed	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained how to prove the covert beacon stopped	3	Periodic magic frame no longer observed

Instructor Notes & Assembly

- The condition byte is the high byte at `0xA245`; the listing shows the halfword `b10a` for `cbz` and the compromised halfword is `b90a`, so the on-disk bytes are `0A B1` for the fix and `0A B9` for the compromise.
- `cbz` branches when the register is zero; `cbnz` branches when it is non-zero. The register here holds the neutralization gate byte, so the semantics are "skip when the gate is clear".
- The beacon interval is `IMPLANT_BEACON_INTERVAL_TICKS ( 8 )`; the magic is the little-endian word `0xEFBEADDE`, which prints as the bytes `DE AD BE EF`.
- Full credit requires both the byte change and a correct statement of the neutralization semantics.
- Note that the anti-debug check at `0x1000A23C` runs before the beacon gate and suppresses the beacon while a probe is attached; this is why a student who simply watches under GDB sees nothing until the trap is defeated.

Task 3: Bug #2 The Logic Bomb (20 points)

Solution

Locate the branch. In `implant_handle_command` (starts at `0x1000A304`) the arming gate is at file offset `0xA30B` (VA `0x1000A30B`). The corrected image is:

```
1000a304 <implant_handle_command>:
1000a304:    b508        push    {r3, lr}
1000a306:    4b0a        ldr     r3, [pc, #40] @ (1000a330 <implant_handle_command+0x2c>)
1000a308:    781a        ldrb    r2, [r3, #0]
1000a30a:    b17a        cbz     r2, 1000a32c <implant_handle_command+0x28>
1000a30c:    b170        cbz     r0, 1000a32c <implant_handle_command+0x28>
1000a30e:    2907        cmp     r1, #7
1000a310:    d90c        bls.n   1000a32c <implant_handle_command+0x28>
1000a312:    2208        movs    r2, #8
1000a314:    4907        ldr     r1, [pc, #28] @ (1000a334 <implant_handle_command+0x30>)
1000a316:    f000 fa23    bl      1000a760 <memcmp>
1000a31a:    b938        cbnz    r0, 1000a32c <implant_handle_command+0x28>
1000a31c:    2101        movs    r1, #1
1000a31e:    4b06        ldr     r3, [pc, #24] @ (1000a338 <implant_handle_command+0x34>)
1000a320:    4a06        ldr     r2, [pc, #24] @ (1000a33c <implant_handle_command+0x38>)
1000a322:    681b        ldr     r3, [r3, #0]
1000a324:    4806        ldr     r0, [pc, #24] @ (1000a340 <implant_handle_command+0x3c>)
1000a326:    3303        adds    r3, #3
1000a328:    6003        str     r3, [r0, #0]
1000a32a:    7011        strb    r1, [r2, #0]
```

The detonation, reached from `implant_tick` when the trigger tick arrives:

```
1000a290:    f000 fa48    bl      1000a724 <valve_close>
```

Instruction decode. `ldr r3, [pc, #40]` loads the command gate byte at `0x20013CEF`, `ldrb r2, [r3, #0]` reads it, and the branch at `0x1000A30A` decides whether the gate is clear. The correct code ignores the arming magic when the gate is clear, so the branch at `0x1000A30A` must be `cbz ( 0xB1 )` to the `0x1000A32C` return. When the gate is set, the code compares the inbound 8 bytes against the arming magic pointer at `0x1000B210` (`memcmp`, `0x1000A316`), and on a match it loads the tick counter, adds the trigger delay (`adds r3, #3`), stores the trigger tick at `0x200136F4`, and sets the armed flag at `0x20013CF0`. The magic bytes on disk at `0x1000B210` are `46 52 4F 53 54 4C 4E 45`, the ASCII string `FROSTLINE`. The condition byte is the high byte at `0x1000A30B`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
0x1000A30B	0xA30B	0xB9	cbnz r2, 0x1000A32C	0xB1	cbz r2, 0x1000A32C

Patch.

File Offset	VA	Original Bytes	Patched Bytes
0xA30B	0x1000A30B	7A B9	7A B1

Why the logic bomb is disarmed. Under the compromised `cbnz`, the arming gate is inverted: the magic comparison runs exactly when the neutralization gate is clear, so `FROSTLNE` arms the bomb. Once armed, `implant_tick` compares the tick counter against the trigger tick and, three ticks later, calls `valve_close` directly. After the patch, `cbz` returns early while the gate is clear, so the magic never reaches `memcmp`, the armed flag is never set, and the valve is never driven by the implant. The bomb does not touch the sealed protocol at all; it calls the actuator underneath it, which is why an authenticated wire is not an authenticated machine.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the arming gate branch at 0x1000A30B	5	Address and function identified
[DOCUMENT] Documented the FROSTLNE arming magic and the 3-tick detonation	5	Correct magic, delay, and detonation path
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so the logic bomb is disarmed	7	Byte 0xB9 changed to 0xB1
[DOCUMENT] Explained that the bomb closes the valve through the actuator path	3	Direct actuator call, no authorization

Instructor Notes & Assembly

- The condition byte is the high byte at 0xA30B; the listing shows the halfword `b17a` for `cbz` and the compromised halfword is `b97a`, so the on-disk bytes are `7A B1` for the fix and `7A B9` for the compromise.
- The arming magic is the 8-byte ASCII string `FROSTLNE` at 0x1000B210, and the trigger delay is `IMPLANT_TRIGGER_DELAY_TICKS ( 3 )`.
- The detonation path calls `valve_close` at 0x1000A724; it does not call `control_handle_frame`, so no tag, no window, and no authorization verdict is consulted.
- Full credit requires both the byte change and a correct statement of the arming and detonation behavior.
- A student may observe the bomb only after defeating the anti-debug; with a probe attached the implant suppresses the arming response.

Task 4: Bug #3 The Persistence Marker (20 points)**Solution**

Locate the branch. The `implant_persist` path is inlined into `implant_tick` (starts at 0x1000A228). After the detonation calls `valve_close`, the code checks the persistence gate and branches at file offset 0xA29F (VA 0x1000A29F). The corrected image is:

```

1000a290:    f000 fa48      bl      1000a724 <valve_close>
1000a294:    2200           movs     r2, #0
1000a296:    4b18           ldr      r3, [pc, #96] @ (1000a2f8 <implant_tick+0xd0>)
1000a298:    702a           strb     r2, [r5, #0]
1000a29a:    781b           ldrb     r3, [r3, #0]
1000a29c:    2b00           cmp      r3, #0
1000a29e:    d0dc           beq.n    1000a25a <implant_tick+0x32>
1000a2a0:    4c16           ldr      r4, [pc, #88] @ (1000a2fc <implant_tick+0xd4>)
1000a2a2:    7823           ldrb     r3, [r4, #0]
1000a2a4:    2b00           cmp      r3, #0
1000a2a6:    d1d8           bne.n    1000a25a <implant_tick+0x32>
1000a2a8:    f3ef 8510      mrs      r5, PRIMASK
1000a2ac:    b672           cpsid    i
1000a2ae:    22ff           movs     r2, #255 @ 0xff
1000a2b0:    f10d 0001      add.w    r0, sp, #1
1000a2b4:    4611           mov      r1, r2
1000a2b6:    f000 fa81      bl      1000a7bc <memset>
1000a2ba:    23c7           movs     r3, #199 @ 0xc7
1000a2bc:    f44f 5180      mov.w    r1, #4096 @ 0x1000
1000a2c0:    480f           ldr      r0, [pc, #60] @ (1000a300 <implant_tick+0xd8>)
1000a2c2:    f88d 3000      strb.w   r3, [sp]
1000a2c6:    f000 fbc7      bl      1000aa58 <__flash_range_erase_veneer>
1000a2ca:    f44f 7280      mov.w    r2, #256 @ 0x100
1000a2ce:    4669           mov      r1, sp
1000a2d0:    480b           ldr      r0, [pc, #44] @ (1000a300 <implant_tick+0xd8>)
1000a2d2:    f000 fba5      bl      1000aa20 <__flash_range_program_veneer>
1000a2d6:    f385 8810      msr      PRIMASK, r5
1000a2da:    2301           movs     r3, #1
1000a2dc:    7023           strb     r3, [r4, #0]

```

Instruction decode. `ldr r3, [pc, #96]` loads the persistence gate byte at `0x20013CF2`, `ldrb r3, [r3, #0]` reads it, and `cmp r3, #0` tests it. The correct code writes no marker when the gate is clear, so the branch at `0x1000A29E` must be `beq (0xD0)` to the `0x1000A25A` return. When the gate is set, a second check loads the write-once latch at `0x20013CF3`; if it is already non-zero the code returns. Otherwise it erases and programs the reserved flash sector at `0x103FF000` (the flash offset literal `0x3FF000` is loaded from `0x1000A300`), storing `0xC7` (`movs r3, #199`) with the Pico SDK `flash_range_erase` and `flash_range_program` calls, then sets the latch at `0x20013CF3` to one so a second detonation never rewrites the marker. The condition byte is the high byte at `0x1000A29F`.

The anti-debug obstacle. Before any of this, `implant_tick` reads CoreDebug DHCSR at `0xE00EDF0`:

```

1000a238:    f8dc 2df0      ldr.w    r2, [ip, #3568] @ 0xdf0
1000a23c:    0791           lsls     r1, r2, #30
1000a23e:    d10c           bne.n    1000a25a <implant_tick+0x32>

```

The shift discards the upper 30 bits and keeps bit 1 (`C_HALT`) and bit 0 (`C_DEBUGEN`) in the carry and zero flags; a non-zero result means a probe is attached, and the branch returns early. `implant_tick` reads the same register again at `0x1000A26C` to stamp the beacon blob.

Defeating the anti-debug. Clear the debug bits in the register as seen by the target, or patch the read in a scratch copy. The register is only a view of debug state, so clearing it makes `implant_debug_attached` return false for that tick. Show the command sequence, not a fabricated transcript:

```

arm-none-eabi-gdb ACT-III.elf
(gdb) target extended-remote /dev/cu.usbmodemXXXX
(gdb) monitor reset halt
(gdb) break implant_tick
(gdb) continue
(gdb) set {unsigned int}0xE00EDF0 = 0
(gdb) continue

```

To observe the write, break on the flash calls at `0x1000A2C6` (`flash_range_erase`) and `0x1000A2D2` (`flash_range_program`) after clearing the debug bits, then read the reserved sector. A second valid method is to patch the `ldr.w` at `0x1000A238` in a scratch copy to load a zero constant instead of `DHCSR` , which removes the trap entirely. The scratch copy is for observation only; the shipped artifact is patched at the defect.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A29F</code>	<code>0xA29F</code>	<code>0xD1</code>	<code>bne.n 0x1000A25A</code>	<code>0xD0</code>	<code>beq.n 0x1000A25A</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA29F</code>	<code>0x1000A29F</code>	DC D1	DC D0

Why no marker is written. Under the compromised `bne` , the persistence gate is inverted: the marker write path is taken exactly when the gate is clear, so the first detonation erases and programs the reserved flash sector `0x103FF000` with `0xC7` . After the patch, `beq` returns while the gate is clear, so the flash erase and program calls at `0x1000A2C6` and `0x1000A2D2` are never reached and the reserved sector stays blank. Because the same patch also disarms the write-once latch path, a second detonation cannot write it either. The marker is a preview of the Act IV lesson: a payload that leaves a durable trace and survives the thing you did to remove it.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the persistence gate branch at <code>0x1000A29F</code>	5	Address and inlined persist path identified
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method
[DOCUMENT & PATCH] Patched <code>0xD1</code> to <code>0xD0</code> so no marker is written to <code>0x103FF000</code>	7	Byte <code>0xD1</code> changed to <code>0xD0</code>
[DOCUMENT] Explained the write-once marker byte <code>0xC7</code> and the Act IV persistence preview	3	Marker, reserved sector, write-once latch, persistence lesson

Instructor Notes & Assembly

- The persistence path is inlined into `implant_tick` ; there is no standalone `implant_persist` symbol in the stripped image.
- The condition byte is the high byte at `0xA29F` ; the listing shows the halfword `d0dc` for `beq` and the compromised halfword is `d1dc` , so the on-disk bytes are `DC D0` for the fix and `DC D1` for the compromise.
- The marker byte is `IMPLANT_MARKER_BYTE` (`0xC7`), the reserved sector is `VALVE_IMPLANT_RESERVE_ADDR` (`0x103FF000`), and the write-once latch is at `0x20013CF3` .
- The `DHCSR` address is `VALVE_IMPLANT_DHCSR_ADDR` (`0xE000EDF0`); bit 0 is `C_DEBUGEN` and bit 1 is `C_HALT` . The anti-debug is identical in both images, so it is an analysis obstacle, not one of the four graded defects.
- Grade the GDB point on a real command sequence and the correct observed code path, not on a memorized register dump. Accept either clearing the bits with GDB or patching the read in a scratch copy.
- A common failure is patching the shipped artifact at `0xA29F` before observing the marker. The order matters: defeat the anti-debug, observe, then patch.

Task 5: Bug #4 The Valve Command Authorization (20 points)

Solution

Locate the branch. In `control_handle_frame` (starts at `0x10007484`) the authorization branch is at file offset `0x74E9` (VA `0x100074E9`). The corrected image is:

```

100074e0:    f000 f89a    bl     10007618 <valve_auth_apply>
100074e4:    4604        mov    r4, r0
100074e6:    2800        cmp    r0, #0
100074e8:    d0e9        beq.n  100074be <control_handle_frame+0x3a>
100074ea:    2101        movs   r1, #1
100074ec:    ea05 0001    and.w  r0, r5, r1
100074f0:    f000 f81c    bl     1000752c <valve_apply_command>

```

Instruction decode. After the sealed frame is opened and the command byte is range-checked, `valve_auth_apply` returns its authorization verdict in `r0`, and `mov r4, r0` preserves it. `cmp r0, #0` tests the verdict, and the branch at `0x100074E8` decides whether the command may reach the actuator. The correct code rejects a failed or replayed authorization, so the branch at `0x100074E8` must be `beq (0xD0)` to the `0x100074BE` reject path, which returns zero. Only a true verdict falls through to `valve_apply_command`. The condition byte is the high byte at `0x100074E9`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x100074E9</code>	<code>0x74E9</code>	<code>0xD1</code>	<code>bne.n 0x100074BE</code>	<code>0xD0</code>	<code>beq.n 0x100074BE</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x74E9</code>	<code>0x100074E9</code>	<code>E9 D1</code>	<code>E9 D0</code>

Why the valve now requires authorization. Under the compromised `bne`, the verdict is inverted: a failed or replayed authorization falls through to `valve_apply_command`, while a genuine authorization branches to the reject path and returns zero. After the patch, `beq` sends a false verdict to the reject path at `0x100074BE`, so an unauthenticated command, a forged command, and a replayed captured command all fail before the actuator is touched. A legitimate authorized command still returns true and falls through to `valve_apply_command`. The authorization verdict is the last gate before motion; inverting it is worse than deleting it, because the machine now acts on exactly the commands it should refuse.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the authorization branch at <code>0x100074E9</code>	5	Address and function identified
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false
[DOCUMENT & PATCH] Patched <code>0xD1</code> to <code>0xD0</code> so unauthorized and replayed commands are rejected	7	Byte <code>0xD1</code> changed to <code>0xD0</code>

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Explained that unauthenticated and replayed valve commands must be rejected	3	The actuator must see only an authorized verdict

Instructor Notes & Assembly

- The condition byte is the high byte at `0x74E9`; the listing shows the halfword `d0e9` for `beq` and the compromised halfword is `d1e9`, so the on-disk bytes are `E9 D0` for the fix and `E9 D1` for the compromise.
- `valve_auth_apply` performs the monotonic anti-replay check and the authenticated-state tag, so this branch is the verdict for both freshness and state integrity.
- Full credit requires the inversion explanation: the compromised build accepts a false verdict and rejects a true one.
- Point out that the rest of the valve command path is correct: the envelope is opened, the command byte is range-checked against the guarded set, and the sequence window and state tag run. Only the verdict seam was broken.
- This is the defect that is a policy defect rather than an implant behavior, and it is the one a defender would fix first in production.

Task 6: Export and Verify (10 points)

Solution

Export. In Ghidra, `File -> Export Program...`, choose `Binary Format`, and save as `ACT-III_fixed.bin`. The shipped image is 50,732 bytes.

Convert.

```
python uf2conv.py ACT-III_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-III_fixed.uf2
```

If `uf2conv.py` is not in the working directory, use the copy shipped with the project repository. The UF2 for ACT-III is 102,400 bytes.

Verify.

```
python scripts/verify_ctf.py
```

Expected result:

```
10/10 checks passed
```

Hardware proof. Flash `ACT-III_fixed.uf2` in BOOTSEL mode and confirm:

- the covert `DE AD BE EF` frame no longer appears every 8 ticks;
- injecting the `FROSTLINE` magic no longer closes the valve;
- the reserved sector at `0x103FF000` stays blank after a trigger;
- an unauthenticated command and a replayed captured command are rejected before the actuator moves;
- a legitimate authorized command still moves the valve, and the emergency stop still forces it closed.

Summary of all patches.

#	Bug	File Offset	Flash Address	Original Byte	Patched Byte
1	The Beacon	0xA245	0x1000A245	B9	B1
2	The Logic Bomb	0xA30B	0x1000A30B	B9	B1
3	The Persistence Marker	0xA29F	0x1000A29F	D1	D0
4	The Valve Command Authorization	0x74E9	0x100074E9	D1	D0

Reflection mapping. The four defects map to real control-system failures:

Defect	Real-world failure
The Beacon	A compromised controller emits covert traffic on a timer that a passive monitor can detect.
The Logic Bomb	A trigger word arms a payload that later actuates the process with no operator in the loop.
The Persistence Marker	A payload writes a durable trace to flash, so it survives removal and can be re-armed.
The Valve Command Authorization	An inverted verdict lets an unauthenticated or replayed command move a physical valve.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[PATCH] Exported ACT-III_fixed.bin from Ghidra	2	Valid patched binary
[PATCH] Converted to ACT-III_fixed.uf2 with the correct base and family	2	--base 0x10000000 --family 0xe48bff59
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four

Instructor Notes & Assembly

- Confirm the exported image differs from ACT-III.bin in exactly the four bytes in the table; scripts/verify_ctf.py checks this and the SHA-256 values.
- Confirm the UF2 conversion used base 0x10000000 and family 0xe48bff59.
- The shipped image is 50,732 bytes; the corrected image must be the same size because every patch is in place.
- Grade the reflection on specificity, not length: each of the four defects should name a concrete control-system consequence.
- Remind students that the anti-debug is not patched out of the shipped artifact; only the four defect bytes change.

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 process sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address 0x27
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	VALVE FAULT, 220 to 330 ohm to GND
Yellow LED	anode	GP17	COMMAND PENDING, 220 to 330 ohm to GND
Green LED	anode	GP18	VALVE NOMINAL, 220 to 330 ohm to GND
Emergency-stop button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Complete Grading Summary

Task	Title	Points
Task 1	Setup and Initial Analysis	10
Task 2	Bug #1 The Beacon	20
Task 3	Bug #2 The Logic Bomb	20
Task 4	Bug #3 The Persistence Marker	20
Task 5	Bug #4 The Valve Command Authorization	20
Task 6	Export and Verify	10

Task	Title	Points
TOTAL		100

Instructor Notes

Safety: Use only the supplied Pico 2, Debug Probe, and firmware. Never connect the exercise to an operational pipeline control system, a pharmaceutical network, a public network, a military system, or a third-party device.

Common Student Mistakes

- Patching the low byte of the branch at `0xA244`, `0xA30A`, `0xA29E`, or `0x74E8` instead of the condition byte at `0xA245`, `0xA30B`, `0xA29F`, or `0x74E9`.
- Reading the beacon gate backwards and believing the corrected build still transmits.
- Searching for a standalone `implant_persist` symbol and missing that it is inlined into `implant_tick`.
- Treating the CoreDebug `DHCSR` anti-debug as a defect and trying to patch it, when it is identical in both images and is an analysis obstacle.
- Patching the shipped artifact before observing the persistence write, so the payload is never demonstrated.
- Reversing the authorization explanation: under the compromise the accept path is taken when the verdict is false.
- Confusing `cbz` and `cbnz` on the two arming gates.
- Forgetting the UF2 conversion or using the wrong family flag.
- Fabricating a GDB session instead of showing the command sequence and the real observed code path.

Partial Credit Guidelines

- Award partial credit for a correct address without the correct byte, or a correct byte without the address.
- Award partial credit for documented before/after bytes without the control-flow explanation, or vice versa.
- Award partial credit for a correct GDB command sequence without a clear statement of the observed code path, or the observation without the commands.
- Award partial credit for a correct anti-debug explanation without a working defeat method, or a working method without the explanation.
- Award no credit for patches that alter any byte outside the four documented offsets, and no credit for a fabricated GDB session.

Appendix: Expected Binary Diff

These offsets are from the compiled image loaded at `0x10000000`.

```

--- ACT-III.bin (compromised)
+++ ACT-III_fixed.bin (corrected)

Offset 0x000074E9: D1 -> D0    (bne.n 0x100074BE -> beq.n 0x100074BE)
Offset 0x0000A245: B9 -> B1    (cbnz r2, 0x1000A24A -> cbz r2, 0x1000A24A)
Offset 0x0000A29F: D1 -> D0    (bne.n 0x1000A25A -> beq.n 0x1000A25A)
Offset 0x0000A30B: B9 -> B1    (cbnz r2, 0x1000A32C -> cbz r2, 0x1000A32C)

```

#	Bug	File Offset	Flash Address	Original Bytes	Patched Bytes
1	The Beacon	0xA245	0x1000A245	0A B9	0A B1
2	The Logic Bomb	0xA30B	0x1000A30B	7A B9	7A B1
3	The Persistence Marker	0xA29F	0x1000A29F	DC D1	DC D0
4	The Valve Command Authorization	0x74E9	0x100074E9	E9 D1	E9 D0

Four defects, four changed bytes in four instructions: the beacon gate, the arming gate, the persistence gate, and the authorization verdict. No other byte in either image differs.